

TECHNICAL ARCHITECTURE & INTERVIEW MASTER GUIDE

मान्यांक: (Maanyankah)

A Social Cinema, Music & Taste Platform with Multi-User Collaborative Filtering, Group Recommender Aggregation, and an AI Cinema Intelligence Agent (चित्राङ्कः).

CREATOR / ENGINEER

Rudra Dwivedi

CORE STACK

Python 3.13, Flask, SQLite

ML / AI DOMAINS

Collaborative Filtering, NLP, LLM

REPOSITORY

Rudra-Dwivedi/Maanyankah

GUIDE CONTENTS & ROADMAP

1. Executive Summary & 30-Sec Elevator Pitch
2. High-Level Architecture & Entity Schema
3. Collaborative Filtering & Group Math Engine
4. Chitrakah (चित्राङ्कः) AI Cinema Agent
5. Curated Collections & Dynamic Filtering
6. Architecture Trade-offs & Engineering Design
7. Scalability & Production Evolution
8. Top 15 Technical Interview Q&A (Cheat Sheet)

1. Executive Summary & High-Level Pitch

The 30-Second Elevator Pitch (Memorize this for the opener!)

🗣️ WHAT YOU SAY TO THE INTERVIEWER

"**Maanyankah (मान्यांकः)** is a full-stack social taste engine and cinema intelligence platform. It bridges the gap between individual entertainment preferences and group decision-making. Using a custom **User-User Collaborative Filtering** recommender with centered cosine similarity, it predicts items you'll love and pairs you with high-affinity taste-twins. Additionally, it implements a **multi-strategy Group Recommender** using Average Satisfaction and Least Misery principles to solve group watch paralysis. Finally, it features **Chitrakah (चित्राङ्कः)**, an intelligent cinema AI agent that parses community rating distributions and viewer sentiment to deliver conversational movie breakdowns."

The Core Problems It Solves

1. **Individual Silos:** Traditional algorithms optimize purely for a single user in isolation, completely ignoring multi-person social contexts (couples, friends, roommates deciding what to watch together).
2. **The Group Compromise Trap (Social Choice Dilemma):** When 4 friends want to watch a movie, picking the highest average rated film often results in one person absolutely hating it (veto dilemma). Maanyankah introduces algorithmic strategy selection (Average vs. Least Misery) to mathematically balance group satisfaction.
3. **Superficial 5-Star Ratings:** A raw average of 3.8/5 hides polarized distributions (e.g., 50% 1-star and 50% 5-star vs 100% 4-stars). Maanyankah's AI Agent parses true community distribution curves and sentiment quotes.

2. High-Level System Architecture & Database Schema

System Layers

1. Presentation Layer (SSR + Progressive AJAX)

Semantic HTML5 & responsive CSS3 custom design tokens supporting Dark/Light modes. Vanilla ES6 AJAX modules for seamless, zero-reload interactions (live rating updates, 1-click collection toggles, avatar DP uploads).

2. Application & Controller Layer

Flask 3.1 application factory with 8 modular Blueprints (`auth` , `items` , `collections` , `recommend` , `agent` , `feed` , `admin` , `settings`). Session authentication and granular Role-Based Access Control (RBAC).

3. Computational & Service Layer

Pure Python mathematical engines: `recommender.py` (Cosine similarity, group scoring), `sentiment.py` (NLP polarity scoring), `movie_agent.py` (heuristic query resolution & AI formatting), and media fetchers.

4. Persistence & Integration Layer

SQLite with Foreign Key enforcement and WAL mode. External API gateways to TMDB, Spotify, Apple Music CDN, and TheSportsDB for automated artwork resolution and metadata.

Database Entity-Relationship Schema

Table	Key Columns & Foreign Keys	Purpose & Architectural Role
<code>users</code>	<code>id</code> (PK), <code>username</code> (UQ), <code>email</code> , <code>password_hash</code> , <code>role</code> , <code>avatar_url</code> , <code>bio</code> , <code>is_active</code>	Stores credentials, profile bio, dynamic avatar photo URL, and RBAC roles (<code>user</code> / <code>admin</code>).
<code>items</code>	<code>id</code> (PK), <code>type</code> , <code>external_id</code> , <code>title</code> , <code>genre_tags</code> , <code>metadata</code> , <code>created_at</code>	Unified catalog storing movies, songs, and sports teams with comma-separated genres and JSON metadata.
<code>user_preferences</code>	<code>id</code> (PK), <code>user_id</code> (FK), <code>item_id</code> (FK), <code>rating</code> (1-5), <code>UNIQUE(user_id, item_id)</code>	Sparse user-item rating matrix. Enforces 1-5 star ratings and atomic upsert behavior.
<code>follows</code>	<code>follower_id</code> (FK), <code>followed_id</code> (FK), <code>PRIMARY KEY(follower_id, followed_id)</code>	Directed social graph enabling taste-based following, activity feeds, and group member selection.
<code>groups</code> & <code>members</code>	<code>groups</code> : <code>id</code> , <code>name</code> , <code>created_by</code> ; <code>members</code> : <code>group_id</code> , <code>user_id</code> (Composite PK)	Entity groups for joint recommender sessions.
<code>posts</code>	<code>id</code> (PK), <code>user_id</code> (FK), <code>item_id</code> (FK), <code>content</code> , <code>sentiment_score</code> , <code>is_pinned</code>	Social micro-reviews tagged to items with automated sentiment polarity scoring.
<code>collections</code>	<code>id</code> (PK), <code>user_id</code> (FK), <code>title</code> , <code>description</code> , <code>is_public</code> , <code>created_at</code> , <code>updated_at</code>	User-curated playlists / watchlists containing both movies and music. Supports public/private flags.
<code>collection_items</code>	<code>id</code> (PK), <code>collection_id</code> (FK), <code>item_id</code> (FK), <code>note</code> , <code>UNIQUE(collection_id, item_id)</code>	Many-to-many join linking items to collections with cascading deletion and optional notes.

3. Machine Learning & Mathematical Algorithms Deep Dive

A. User–User Collaborative Filtering Engine (Taste Matching)

The recommender engine discovers individuals with identical aesthetic affinities across films and music. Rather than relying on rigid categorical metadata, it compares empirical rating distributions across shared consumed items.

▶ CENTERED COSINE SIMILARITY (PEARSON BASELINE) FORMULATION

For two users u and v who have rated a shared intersection of items $I_{uv} = I_u \cap I_v$:

$$s(u, v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u) \cdot (r_{v,i} - \bar{r}_v)}{\sqrt{[\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2]} \cdot \sqrt{[\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2]}}$$

Where $r_{u,i}$ is user u 's rating for item i , and $\bar{r}_u$ is user u 's mean rating across all rated items. Mean-centering normalizes for user rating biases (optimistic raters who always give 4–5 stars vs critical raters who give 2–3 stars).

Predicted Rating Formulation: To recommend an unrated item i to target user u , we compute a similarity-weighted average over the top- K neighborhood N of similar users who have rated item i :

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N} s(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N} |s(u, v)|}$$

B. Multi–Strategy Group Decision Engine (Solving Compromise Traps)

When a group of users $G = \{u_1, u_2, \dots, u_m\}$ wants to select an entertainment item, Maanyankah executes joint preference aggregation under two distinct social choice strategies:

Strategy	Mathematical Objective	Ideal Scenario & Behavioral Rationale
1. Average Satisfaction Strategy (Utilitarian)	Score_{avg}(i, G) = $\frac{1}{ G } \sum_{u \in G} r_{u,i}$ Maximizes aggregate utility across all members.	Best for large parties / general crowds: Focuses on collective total happiness. Can be susceptible to the "tyranny of the majority" where one minority user intensely dislikes the winner.
2. Least Misery Strategy (Egalitarian)	Score_{LM}(i, G) = $\min_{u \in G} \{ r_{u,i} \}$ Maximizes the utility of the least satisfied member.	Best for intimate groups / date nights / roommates: Prevents any single person from having a terrible evening. A movie rated [4, 4, 4] beats a movie rated [5, 5, 1] because its minimum satisfaction is 4 vs 1.

4. Chitrakah (चित्राङ्कः) AI Cinema & Ratings Agent

Chitrakah is an intelligent cinema co-pilot directly integrated into both a dedicated studio cockpit (`/agent/`) and a persistent floating widget (`_floating_agent.html`).

Natural Language Query Parsing

Analyzes incoming queries via regex entity recognition and semantic pattern matching. Distinguishes between film deep-dives ("Tell me about *Inception*"), community ranking requests ("Highest rated movies"), and genre discovery queries ("Recommend a sci-fi thriller").

Empirical Rating Distribution Analysis

Rather than quoting an external IMDb score, Chitrakah dynamically queries SQLite to calculate exact community score distributions (1★ to 5★ curves) and synthesizes genuine viewer quotes logged by community members.

💡 HOW TO EXPLAIN CHITRAKAH TO AN INTERVIEWER

"Chitrakah is designed with a layered architecture: when external LLM API keys (Gemini / OpenAI) are provided via our encrypted UI Settings store, it invokes prompt-engineered generative responses grounded by our live SQL database context. In offline or local development mode, a deterministic heuristic engine analyzes rating distributions, genre tags, and viewer review sentiment to provide instant, zero-latency cinema answers without incurring external API costs."

5. Curated Collections & Dynamic Genre Filtering

A. Curated Collections System (Playlists & Watchlists)

- **Cross-Media Hybrid Playlists:** Users can bundle films and music together (e.g., "Cyberpunk Cinema & Synthwave Beats") with custom descriptions, item notes, and public/private visibility toggles.
- **2x2 Artwork Mosaic Previews:** Automatically generates visual mosaic thumbnails on collection cards by stitching up to 4 high-definition movie theatrical posters and square song album covers.
- **1-Click Quick Add Modal:** An interactive AJAX modal (`_add_to_collection_modal.html`) accessible from every movie and song card on the Browse page. Features immediate checkmark state toggling and inline creation of new collections without page reloads.

B. Dynamic Genre-Wise Filtering

- **Multi-Dimensional Tag Parsing:** Items store comma-separated genre tags (e.g., `sci-fi,thriller,action`). The server extracts distinct genres dynamically from the catalog and computes live item counts.
- **Exact Delimited SQL Matching:** Eliminates substring false-positives (e.g., preventing "rock" from matching "hard-rock") using normalized bounding delimiters:

```
WHERE (',' || LOWER(REPLACE(i.genre_tags, ' ', '')) || ',') LIKE '%,sci-fi,%'
```

- **Interactive UI Controls:** Horizontal scrollable pill bar, active filter alert badge with 1-click reset, and clickable hashtag chips directly on cards.

6. Software Engineering Best Practices & Architecture Decisions

Design Decision	What Was Chosen	Technical Justification (Why this over alternatives?)
Architecture Pattern	Server-Side Rendering (SSR) with Jinja2 + Progressive AJAX Enhancement	Eliminated massive frontend bundle sizes, slow hydration, and SEO penalties of heavy SPAs (React/Angular). Page loads in <100ms with snappy AJAX micro-interactions where needed.
Database Engine	SQLite 3 with Foreign Keys & WAL (Write-Ahead Logging) Mode	Zero-config, embedded, and single-file portability. WAL mode allows concurrent readers without blocking writers. Perfect for single-node deployment on Railway/Fly.io.
Media Fallback Pipeline	High-res multi-tier CDN caching (IMDb 1000px, Apple Music, TheSportsDB)	Items automatically fallback to clean vector placeholders when artwork is unavailable, eliminating broken images and layout shifts (CLS).
State Management	Flask Request-Scoped Context (<code>g</code>) & Client-Side Cookie Sessions	Ensures thread-safe, decoupled database connections per request while preventing connection leaks via <code>teardown_appcontext</code> .

7. Scalability & System Evolution (How would you scale this?)

In an interview, you will almost certainly be asked: *"This works great for 1,000 users. How would you scale it to 1,000,000 users?"* Here is the exact roadmap:

1. Recommender Precomputation (Offline Batching)

Computing user-user cosine similarity on the fly becomes $O(N \cdot M)$ at scale. We transition to offline batch matrix factorization (TruncatedSVD / Alternating Least Squares) run on Celery/Redis workers overnight, storing precomputed top-50 recommendations in Redis key-value stores for $O(1)$ lookups.

2. Database Migration & Read Replicas

Migrate SQLite to PostgreSQL with PgBouncer connection pooling. Implement primary-replica replication: all rating and post writes go to the primary node, while browse and recommender queries distribute across read replicas.

3. Approximate Nearest Neighbors (ANN)

Replace exact cosine similarity matrix scans with vector embeddings (e.g., using FAISS or pgvector with HNSW index) to discover similar taste profiles in sub-millisecond logarithmic time $O(\log N)$.

4. Edge Media Caching (CDN & Object Storage)

Offload external image fetching to AWS S3 / Cloudflare R2 with Cloudflare CDN caching, serving responsive WebP/AVIF formats at the edge to achieve 99.9% cache hit ratios.

8. Top 15 Technical Interview Questions & Perfect Answers (Part 1)

Q1 What was the primary motivation behind building Maanyankah?

Entertainment platforms are fundamentally designed for individual consumption. However, media consumption is deeply social. I wanted to build a unified platform that solves social coordination problems: discovering friends who share niche tastes across multiple mediums (film and music), curating hybrid collections, and mathematically solving group decision deadlocks using multi-strategy collaborative choice theory.

Q2 Why did you use Centered Cosine Similarity instead of standard Euclidean distance or raw Cosine?

Raw cosine similarity does not account for user rating bias. For example, a generous user might rate good movies as 5 and average movies as 4, whereas a critical user might rate good movies as 3 and average movies as 2. By subtracting the user's mean rating ($\bar{r}_u$), centered cosine normalizes their psychological baseline, effectively converting cosine similarity into the Pearson Correlation Coefficient over common items.

Q3 How does your system address the Recommender Cold-Start Problem?

For new users with zero ratings, collaborative filtering lacks historical data. Maanyankah employs a hybrid mitigation strategy: new users are presented with popular, high-consensus catalog items on the Browse page; as soon as a user rates 3–5 items, their cosine similarity vector activates. For items with zero ratings, we fallback to genre tag clustering and community ranking.

Q4 Explain the difference between Average Satisfaction and Least Misery in your group recommender.

Average Satisfaction calculates the arithmetic mean of ratings across all group members. It maximizes total social welfare but can leave a vocal minority extremely unhappy. **Least Misery** takes the minimum rating for each item across all members and ranks by this minimum. It guarantees that the chosen item minimizes the worst-case individual dissatisfaction, making it ideal for date nights or small groups where consensus is critical.

Q5 How does Chitrakah (चित्राङ्कः) analyze rating curves rather than just averages?

A film with an average score of 3.5 could either be universally mediocre (everyone rated 3 or 4) or highly polarized (half gave 1 star, half gave 5 stars). Chitrakah queries the SQLite database for the raw distribution of star counts (1★ through 5★) and calculates the variance. If the bimodal variance exceeds a threshold, it highlights that the film has a polarized reception and extracts top quotes from both ends of the spectrum.

Q6 How do you prevent race conditions or duplicate entries in the Collection toggle API?

At the database layer, the `collection_items` table enforces a strict `UNIQUE(collection_id, item_id)` composite constraint. In the application controller, the toggle endpoint checks for existence within a transactional SQLite session: if present, it deletes the row; if absent, it inserts it. Foreign keys are enabled with cascading deletion so orphaned items are never left behind.

Q7 How does the 2x2 artwork mosaic generation work for Collections?

When rendering collection cards on `/collections/`, the server queries up to 4 distinct items inside that collection. In CSS, we implemented a responsive grid with `grid-template-columns: 1fr 1fr; grid-template-rows: 1fr 1fr;`. Depending on whether the collection contains 1, 2, 3, or 4+ items, CSS rules adapt the mosaic layout seamlessly, displaying high-res theatrical posters and album art with aspect-ratio preservation.

8. Top 15 Technical Interview Questions & Perfect Answers (Part 2)

Q8 Why did you choose Server-Side Rendering (SSR) with Jinja2 instead of a React or Next.js SPA?

For a content and media platform, First Contentful Paint (FCP) and low client memory footprint are paramount. Jinja2 renders complete HTML on the server in <15ms. To maintain a modern app experience, we selectively layered progressive enhancement with Vanilla ES6 AJAX for dynamic actions (liking posts, toggling collections, editing DP photos) without the bundle overhead, hydration delay, and state complexity of a full SPA.

Q9 How does sentiment analysis work on user posts and feed activity?

When a user shares a review or comment on a movie or song, the backend passes the text through our sentiment analysis pipeline (`sentiment.py`). It computes a normalized compound polarity score between -1.0 (strongly negative) and +1.0 (strongly positive). We then classify the review into visual badges (Positive, Neutral, Critical), allowing fans to quickly gauge the sentiment tone of community discussions.

Q10 How is user authentication and session security implemented?

Passwords are never stored in plain text; they are hashed using Werkzeug's secure implementation of Scrypt / PBKDF2 with unique cryptographic salts. User sessions are signed using Flask's HMAC-SHA256 session cookie mechanism. We implemented `@login_required` and role verification decorators to prevent Broken Access Control (OWASP Top 10).

Q11 How do you handle exact genre tag filtering in SQL without false positives?

Items store tags as comma-separated values (e.g., `sci-fi,thriller,action`). A naive `LIKE '%rock%'` would incorrectly match `hard-rock` . In Maanyankah, we wrap both the column and the search query in bounding delimiters: `WHERE (' || LOWER(REPLACE(i.genre_tags, ' ', '')) || ',') LIKE '%,rock,%'` . This guarantees 100% exact tag matching directly in SQL.

Q12 What is the role of SQLite WAL mode in your deployment?

Standard SQLite locks the entire database file during writes, causing read latency spikes under concurrent traffic. By enabling Write-Ahead Logging (`PRAGMA journal_mode = WAL;`), changes are appended to a separate `-wal` file. This allows readers to continue reading unblocked while a write transaction commits, drastically improving concurrency on single-instance web servers.

Q13 How does the in-place dynamic profile photo (DP) and bio editor work?

The profile page includes an interactive modal (`_profile_modal.html`). Users can either upload an image file (PNG/JPG) or paste an image URL. Upon submission, an asynchronous AJAX request sends a `FormData` payload to `/auth/profile` . The server validates image dimensions and security headers, updates the user's record, and returns a JSON response that updates the DOM avatar and bio instantly without a page reload.

Q14 What testing methodology was used to verify platform reliability?

We built automated end-to-end Python test harnesses (`verify_features.py` , `verify_collections_and_genres.py`) using Flask's test client and isolated database fixtures. The tests simulate multi-user sessions, verify collaborative filtering matrix integrity, assert HTTP 200/403/404 response codes, validate JSON API schemas, and enforce privacy authorization checks on private collections.

Q15 If you had another 2 weeks, what advanced feature would you implement next?

I would implement a hybrid Deep Learning recommender using Two-Tower Neural Networks (one user tower embedding user taste history, and one item tower embedding film/song audio & synopsis embeddings generated via sentence-transformers). This would unify collaborative filtering with content embeddings to solve zero-shot cold start recommendations for newly released media.